



COMP 4300

Intro to Game Programming

Lecture 21

Particle Systems

Vertex Arrays

ECS and Particles

- Main ECS philosophy is that everything in our game is an Entity with Components
- ... except when it shouldn't be ...
- It turns out that particle systems are a case where we would have objects in our game that are not necessarily Entities
- Let's explore why and how

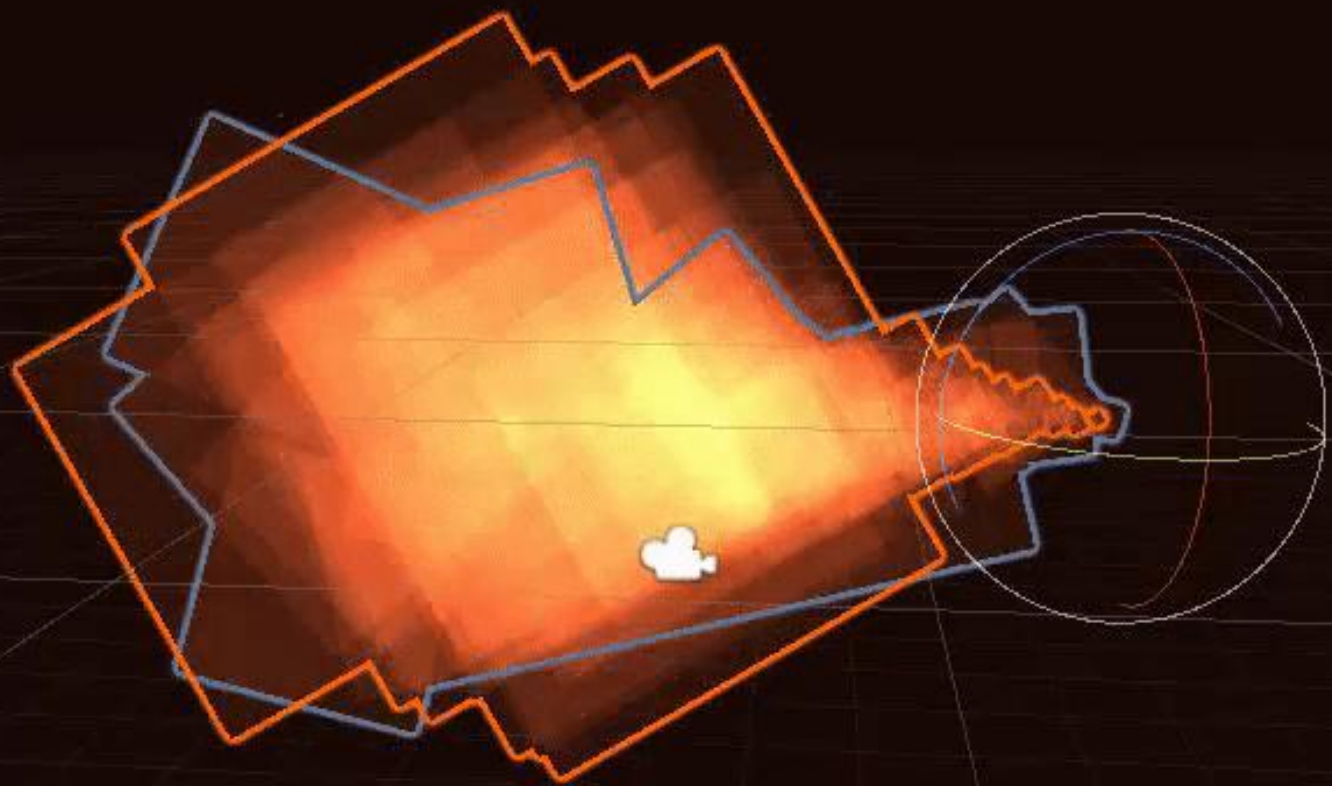
Particle Systems

- It may be interesting to have a system in our game which is composed of **particles**
- Many small (similar looking) particles come together to form a complex system
- The behavior of a particle is usually **simple**
- The lifespan of a particle is usually **short**
- There can be **MANY** particles...

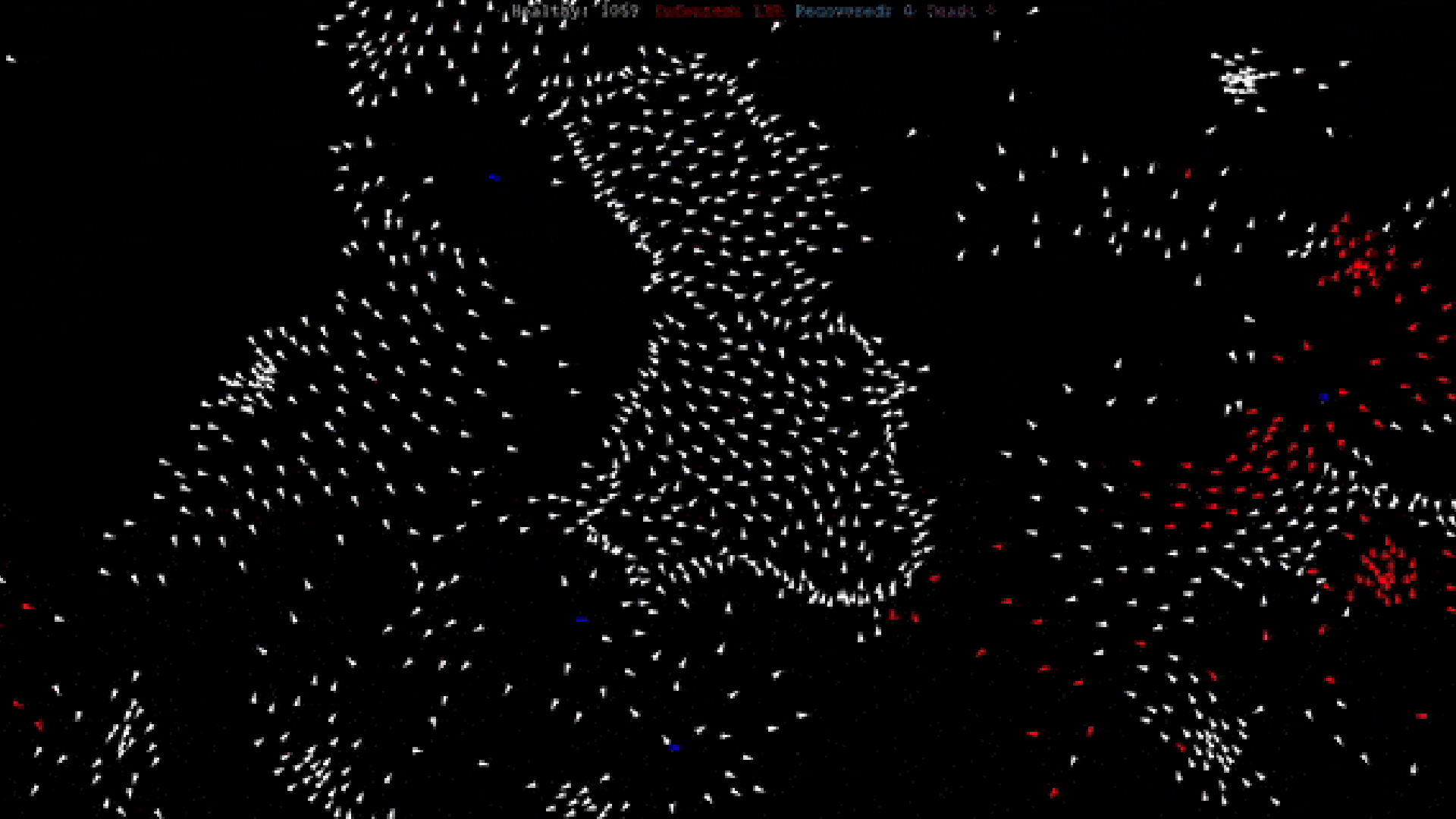












Particle Systems

- Most modern game engines come with solutions for building particle systems
- User specifies the behavior and look of the particles, engine handles the details
- By varying parameters, users can get the desired effects from the particle system

```
let size = CGSizeMake(width: 859.0, height: 1116.0)
let host = UIView(frame: CGRectMake(0.0, y: 0.0, width:
size.width, height: size.height))
self.view.addSubview(host) // Add to parent view
```

```
let particlesLayer = CALayer()
particlesLayer.frame = CGRectMake(0.0, y: 0.0, width:
size.width, height: size.height)
host.layer.addSublayer(particlesLayer)
host.layer.masksToBounds = true // constrain particles to
host
```

```
particlesLayer.backgroundColor = UIColor(red: 0.0/255.0,
green: 0.0/255.0, blue: 0.0/255.0, alpha: 1.0).CGColor
particlesLayer.emitterShape = .circle
particlesLayer.emitterPosition = CGPointMake(346.7, y: 374.3)
particlesLayer.emitterSize = CGSizeMake(width: 333.0, height:
432.0)
particlesLayer.emitterMode = .outline
particlesLayer.renderMode = .oldestFirst
```

```
/* Import texture -- Export texture */
let image1 = UIImage(named: "snow").CGImage
```

```
let cell1 = CAEmitterCell()
cell1.contents = image1
cell1.name = "cell1"
cell1.birthRate = 40.0
cell1.lifetime = 55.0
cell1.lifetimeRange = 0.0
cell1.velocity = -30.0
cell1.velocityRange = -70.0
cell1.xAcceleration = 5.0
cell1.yAcceleration = 30.0
cell1.emissionLatitude = 0.0 * (.pi / 180.0) // to radians
cell1.emissionLongitude = 0.0 * (.pi / 180.0)
cell1.emissionRange = 180.0 * (.pi / 180.0)
cell1.spin = -30.0 * (.pi / 180.0)
cell1.spinRange = 57.3 * (.pi / 180.0)
cell1.scale = 0.06
cell1.scaleRange = 0.3
cell1.scaleSpeed = 0.0
cell1.alphaRange = 0.0
cell1.alphaSpeed = 0.0
cell1.color = UIColor(red: 255.0/255.0, green: 255.0/255.0,
blue: 255.0/255.0, alpha: 1.0).CGColor
cell1.redRange = 0.0
cell1.greenRange = 0.0
cell1.blueRange = 0.0
```



CA Emitter Layer Editor

Emulation ☒ Restart on new values

Emitter Layer

Background

Position
X Y

Size
Width Height

Emitter Mode

Render Mode

Emitter Cells

Sub cells

Name

Texture

Emitter
Birth rate

Lifetime
Start Range

Velocity
Start Range

Acceleration
X Y

Emission
Latitude Longitude

Emission Range

Spin
Start Range

Scale
Start

Range Speed

Alpha
Range Speed

Color

```

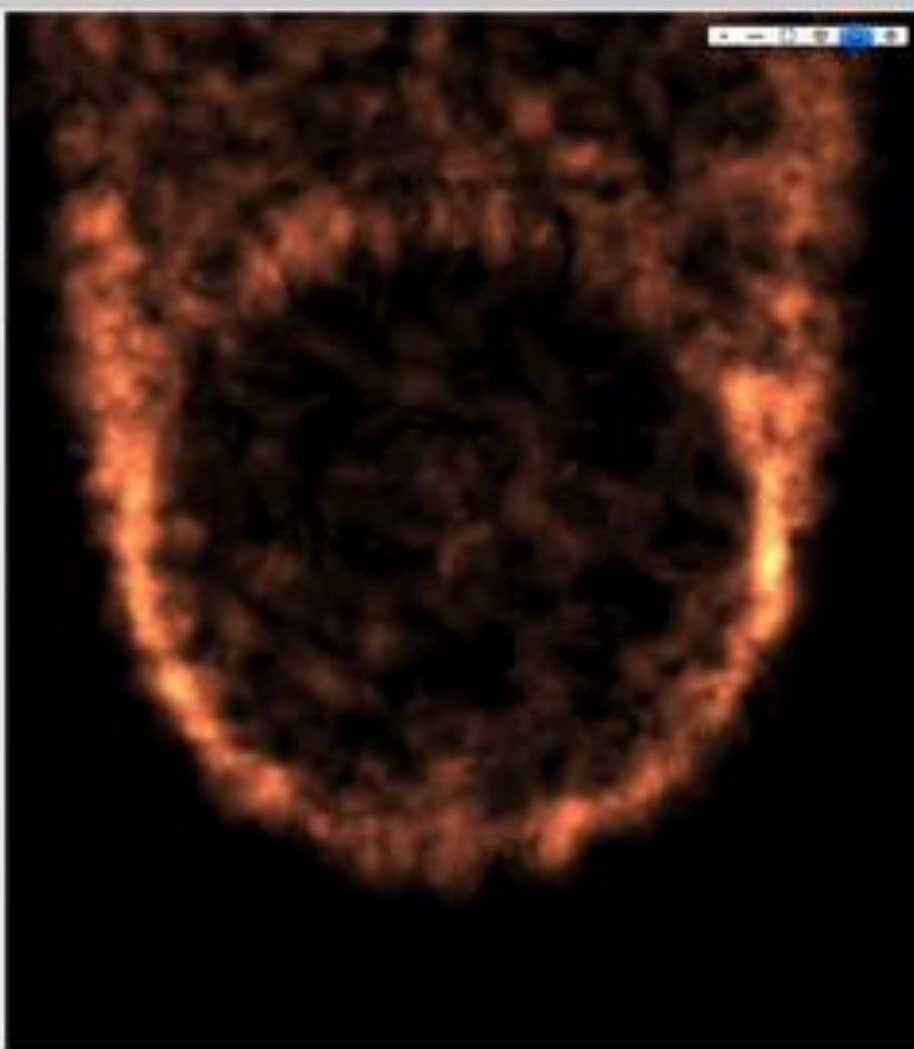
cell1 = InteractiveCell()
cell1.contents = Image()
cell1.name = "Cell1"
cell1.birthdate = 11.0
cell1.lifeSpan = 10.0
cell1.velocity = 10.0
cell1.velocityAngle = 10.0
cell1.rotationRate = 0.0
cell1.rotationAngle = -100.0
cell1.rotationVelocity = 0.0 * 1.0 / 100.0 // 10 radians
cell1.rotationAngle = 0.0 * 1.0 / 100.0
cell1.rotationRate = 0.0 * 1.0 / 100.0
cell1.size = 17.5 * 1.0 / 100.0
cell1.radius = 17.5 * 1.0 / 100.0
cell1.scale = 0.1
cell1.colorRange = 0.0
cell1.colorAngle = 0.1
cell1.alphaRange = 0.0
cell1.alphaAngle = 0.1
cell1.alphaRange = 0.12
cell1.color = diffusedRed(10.0/100.0, green(10.0/100.0, blue(10.0/100.0,
alpha: 0.1), color
cell1.radius = 0.0
cell1.greenRange = 0.0
cell1.blueRange = 0.0

// Image Range = Image Range, //
Image = diffusedRed(10.0/100.0, green(10.0/100.0, blue(10.0/100.0,
alpha: 0.1), color

cell2 = InteractiveCell()
cell2.contents = Image()
cell2.name = "Cell2"
cell2.birthdate = 11.0
cell2.lifeSpan = 11.0
cell2.velocity = 10.0
cell2.velocityAngle = 10.0
cell2.rotationRate = 0.0
cell2.rotationAngle = -100.0
cell2.rotationVelocity = 0.0 * 1.0 / 100.0 // 10 radians
cell2.rotationAngle = 0.0 * 1.0 / 100.0
cell2.rotationRate = 0.0 * 1.0 / 100.0
cell2.size = 17.5 * 1.0 / 100.0
cell2.radius = 17.5 * 1.0 / 100.0
cell2.scale = 0.01
cell2.colorRange = 0.0
cell2.colorAngle = 0.0
cell2.alphaRange = 0.0
cell2.alphaAngle = 0.0
cell2.color = diffusedRed(10.0/100.0, green(10.0/100.0, blue(10.0/100.0,
alpha: 0.1), color
cell2.radius = 0.0
cell2.greenRange = 0.0
cell2.blueRange = 0.0

particleLayer.addParticle = [cell1, cell2]

```



Cell Editor Layer Editor

Execution:

Cell Editor Layer

Properties:

Position:

Size:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

Color:

Alpha:

Radius:

Scale:

Velocity:

Velocity Angle:

Rotation Rate:

Rotation Angle:

Rotation Velocity:

Rotation Angle:

Rotation Rate:

Size:

Radius:

Scale:

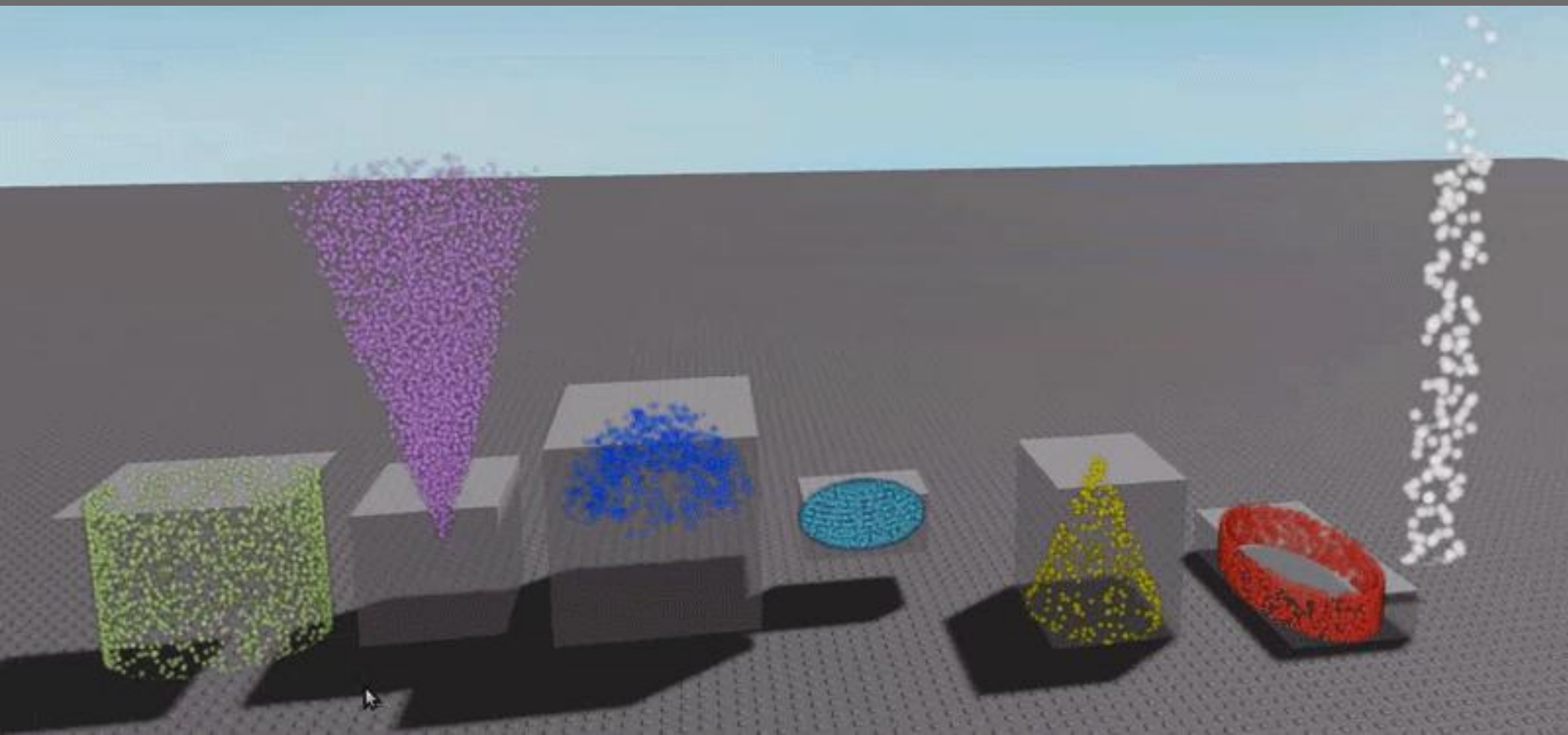
Color:

Why not ECS for Particles?

- It is possible to implement particles using our current ECS system, but inefficient
- Hundreds of thousands of particles may need to be created and destroyed per sec
- Particles will usually not need all of the associated components of our ECS system
- Each particle stores minimal data

Particle System Architecture

- **Memory Field**: Contiguous block of memory allocated for particle storage
- **Renderer**: Define how particles drawn
- **Initializer**: Define initial particle data
- **Emitter**: Define how particles created
- **Forces**: Define physics of the particles
- Possibly others, depending on complexity



Drawing Particles Quickly

- So far in this course we have been drawing each Entity with its own draw() call
- Separate draw calls from the CPU to the GPU can be very time consuming
- Ideally we would want a way to send all data from CPU to GPU and tell the GPU to draw all the particles in one instruction

Grocery Analogy



Vertex Arrays

- Vertex Arrays are a way to send a single GPU instruction to draw many things
- There are two parts to a vertex array draw
- Vertex Data: Position / Color of vertices
- Primitive Type: Type of shapes to draw
- We will use a Vertex Array to draw all of our particles in one draw command

SFML Vertex Array Tutorial

<https://www.sfml-dev.org/tutorials/2.5/graphics-vertex-array.php>

Making A Particle System

- We will create a class called ParticleSystem
- This class will have the following:
 - Contiguous vector of Particle data
Velocity, Lifetime, Speed, Steering, etc
 - VertexArray position/color data
 - Some data about the game world / window
 - Functions that let us initialize, reset, update and draw the particles

WE'LL DO IT LIVE

I'LL WRITE IT AND WE'LL DO IT LIVE